

Comparative Study of Single-Source Shortest Path Algorithms

Ahmet Alp Keleş

Dijkstra | *Bellman-Ford* | *BMSSP/DMMSY*

Datasets: DIMACS Road Networks (NY, COL, NE) | Language: C++17 | Platform: WSL (Ubuntu/Linux)

Abstract

This study presents an empirical comparison of three single-source shortest path (SSSP) algorithms-Dijkstra, Bellman-Ford, and a practical BMSSP/DMMSY variant-on three directed DIMACS road-network datasets of increasing size (NY, COL, and NE). Each dataset was evaluated using five distributed source nodes with three repeated runs per source. Runtime was measured with `std::chrono`, while memory usage was estimated analytically and cross-checked with `ru_maxrss` under WSL. The results show that Dijkstra and BMSSP/DMMSY operate in nearly the same runtime band, whereas Bellman-Ford becomes dramatically slower at scale. BMSSP/DMMSY remains competitive in speed but requires substantially higher working memory than Dijkstra.

1. Introduction

The single-source shortest path (SSSP) problem asks for the shortest distance from one source vertex to all other reachable vertices in a directed weighted graph. This report compares Dijkstra, Bellman-Ford, and a practical BMSSP/DMMSY implementation using real DIMACS road-network benchmarks. The comparison focuses on runtime, approximate memory usage, and scalability.

2. Algorithms

Dijkstra: For directed graphs with non-negative edge weights, Dijkstra maintains a min-priority queue of (distance, vertex) pairs, repeatedly extracts the closest unfinished vertex, skips stale entries, and relaxes outgoing edges. The $O((V+E) \log V)$ complexity arises because each vertex is extracted once - $O(V \log V)$ - and each edge triggers at most one decrease-key operation - $O(E \log V)$. In our implementation, the graph is stored as an adjacency list and distances are tracked in a long long array. The non-negative weight requirement is fundamental: without it, a finalized vertex could later receive a shorter path, breaking the greedy invariant. Complexity: $O((V+E) \log V)$.

Bellman-Ford: This algorithm relaxes all edges up to $V-1$ times, guaranteeing correctness because any simple shortest path visits at most $V-1$ edges. The $O(VE)$ bound comes directly from $V-1$ passes each scanning all E edges. In practice, early termination - halting when a full pass produces no update - reduces the effective iteration count significantly on sparse road networks, where convergence is often reached in far fewer than $V-1$ passes. Our implementation uses the same adjacency-list representation as Dijkstra for structural fairness and performs a final pass for negative-cycle detection. Complexity: $O(VE)$.

BMSSP/DMMSY: This practical method computes SSSP on directed non-negative graphs using CSR storage together with a fast 4-ary heap and recursive workspace buffers. The theoretical $O(m \log^{2/3} n)$ bound of the full DMMSY algorithm (Duan et al., STOC 2025) breaks the classical $O(m + n \log n)$ sorting barrier by exploiting atomic heaps and recursive subproblem decomposition. In practice on road networks, however, this advantage is not observable: sparse, low-diameter graphs already give Dijkstra near-optimal cache behavior, and the theoretical gain requires denser or adversarially structured inputs to manifest. The evaluated repository is a practical variant rather than a verbatim realization of the paper's full theoretical machinery, so results should be read as practical performance evidence rather than confirmation of the exact asymptotic bound. Complexity: $O(m \log^{2/3} n)$.

3. Experimental Setup

All algorithms were compiled with `g++ -O2` and executed under WSL to enable Linux-based memory measurement through `getrusage()` and `ru_maxrss`. Dijkstra and Bellman-Ford were implemented from scratch in C++, whereas BMSSP/DMMSY was evaluated using an adapted reference implementation [3]. All three implementations were cross-validated and produced identical shortest-path distances on the tested source nodes. For each dataset, five source nodes were selected by the rule approximately $n/10$, $3n/10$, $n/2$, $7n/10$, and $9n/10$ to distribute the tests across the graph.

Dataset	Vertices (n)	Edges (m)	Description
NY	264,346	733,846	New York City road network
COL	435,666	1,057,066	Colorado road network
NE	1,524,453	3,897,636	Northeast USA road network

The dataset scale increases substantially from NY to NE, expanding from 264,346 to 1,524,453 vertices and from 733,846 to 3,897,636 directed edges; this size spread makes the scalability comparison meaningful.

Dataset	S1	S2	S3	S4	S5
NY	26,435	79,304	132,174	185,043	237,912
COL	43,567	130,700	217,834	304,967	391,100
NE	152,446	457,336	762,227	1,067,118	1,372,008

Table 1. Source nodes used for repeated experiments (1-based indexing).

4. Results

4.1 Average Runtime Summary

Algorithm	NY (s)	COL (s)	NE (s)
Dijkstra	0.0357	0.0579	0.2326
Bellman-Ford	1.2170	3.6139	34.243
BMSSP/DMMSY	0.0381	0.0594	0.2441

Runtime Comparison — SSSP Algorithms

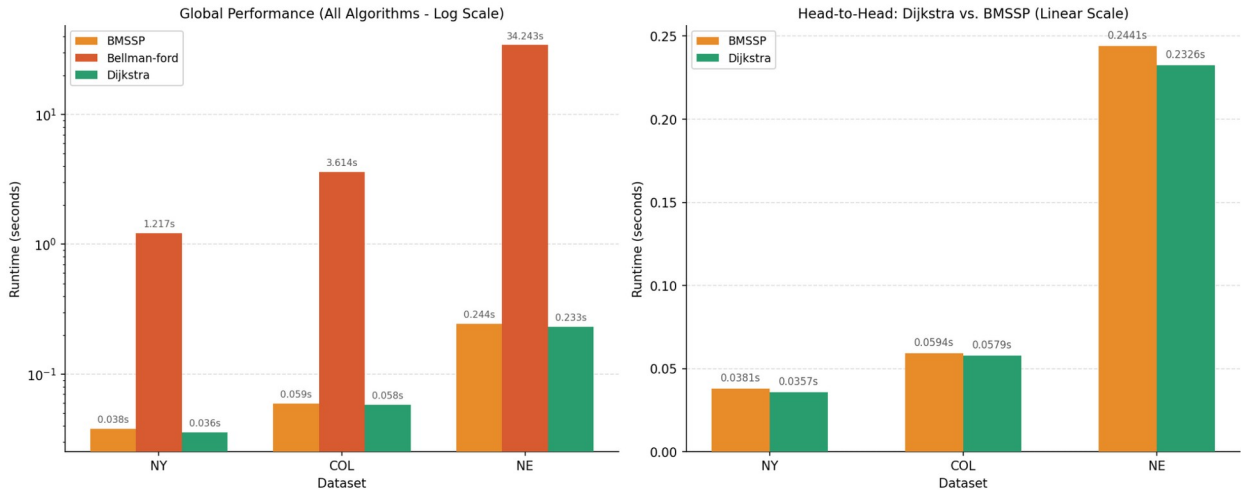


Figure 1. Runtime comparison (left: log scale for all algorithms; right: linear head-to-head of Dijkstra and BMSSP).

As shown in Figure 1, Dijkstra and BMSSP stay in nearly the same runtime band (0.0357-0.2441 s), while Bellman-Ford rises from 1.2170 s on NY to 34.2428 s on NE; on NE, Bellman-Ford is about 147x slower than Dijkstra and 140x slower than BMSSP.

4.2 Approximate Memory Usage

Algorithm	NY-Graph	NY-Work	NY-Total	NE-Graph	NE-Work	NE-Total
Dijkstra	11.6 MB	2.03 MB	13.7 MB	64.6 MB	11.65 MB	76.3 MB
Bellman-Ford	11.6 MB	2.02 MB	13.7 MB	64.6 MB	11.63 MB	76.3 MB
BMSSP/DMMSY	12.2 MB	13.1 MB	25.3 MB	65.3 MB	75.6 MB	140.9 MB

Memory Usage Breakdown: Graph Storage vs. Working Memory

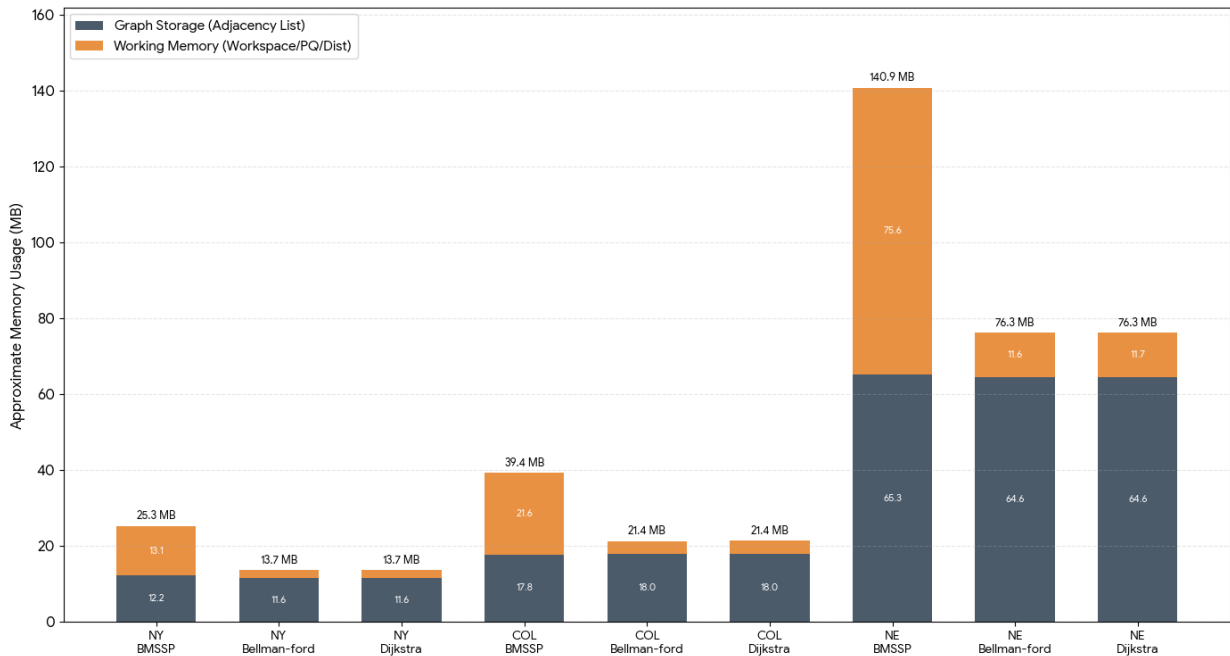


Figure 2. Approximate memory breakdown into graph storage and algorithm working memory.

Figure 2 highlights the main trade-off of the study: BMSSP remains fast, but its total approximate memory is consistently about 1.85x Dijkstra's-25.32 vs. 13.68 MB on NY, 39.40 vs. 21.38 MB on COL, and 140.89 vs. 76.94 MB on NE.

4.3 Correctness and Scalability

Cross-validation confirmed identical shortest-path distances across all three implementations on the tested source nodes.

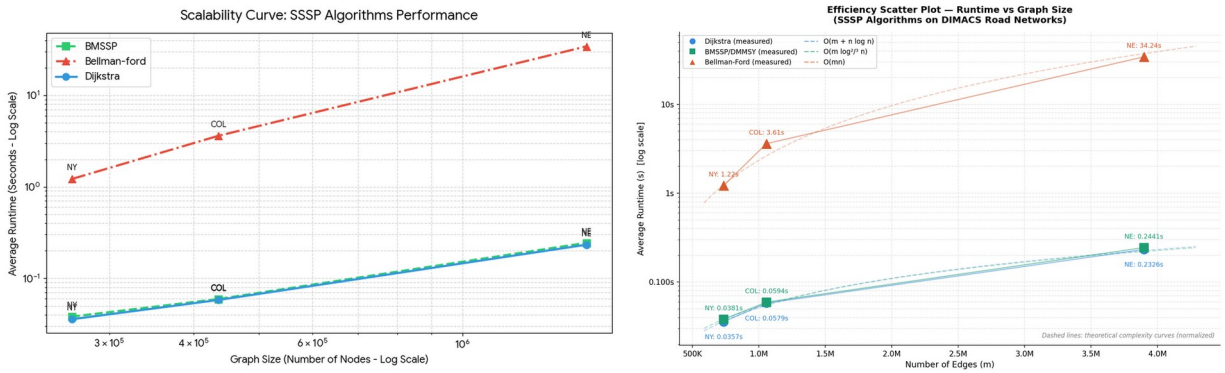


Figure 3. Scalability and efficiency: average runtime against graph size, plus measured points over normalized theoretical trend lines. As shown in Figure 3, Dijkstra and BMSSP exhibit near-parallel growth as graph size increases, whereas Bellman-Ford diverges sharply; in practice, Dijkstra and BMSSP remain usable at larger scales, while Bellman-Ford becomes rapidly inefficient.

4.4 Source-Node Sensitivity (NE Dataset)

Source Sensitivity / Stability Analysis — NE Dataset (n=1,524,453 | m=3,897,636)

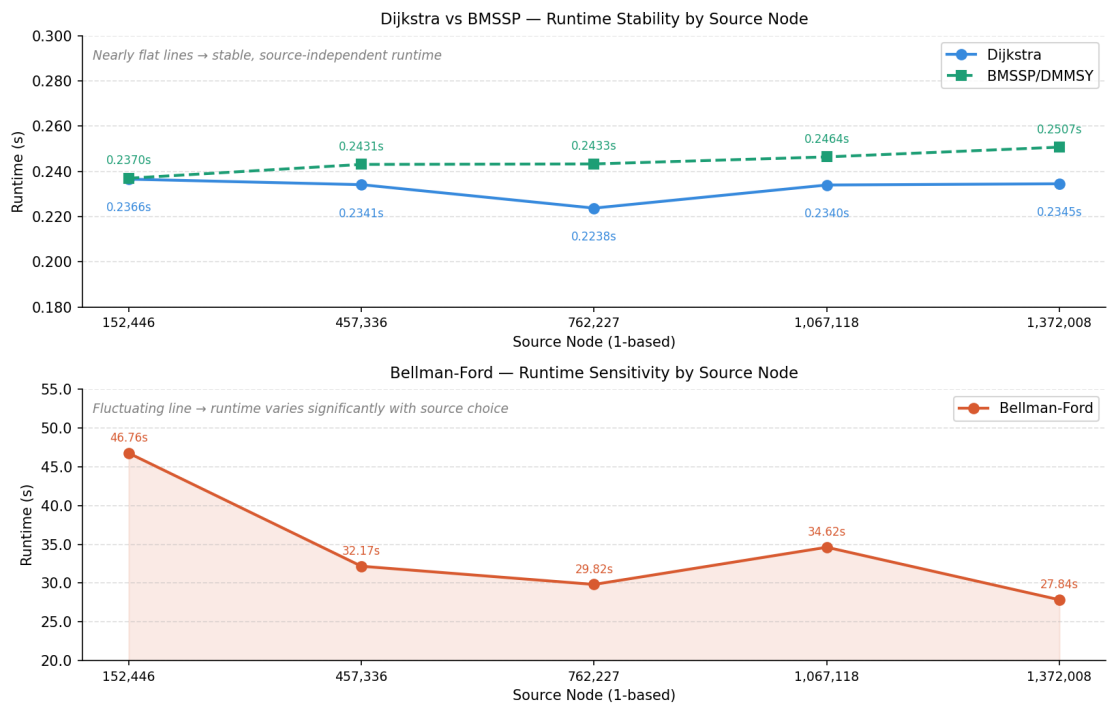


Figure 4. Source sensitivity on NE: Dijkstra and BMSSP remain almost flat, while Bellman-Ford varies strongly with source position. As shown in Figure 4, Dijkstra varies only slightly across sources on NE, BMSSP is similarly stable, and Bellman-Ford fluctuates from about 27.8 s to 46.8 s depending on the source node and the number of effective relaxation passes.

5. Discussion

Bellman-Ford is by far the slowest algorithm. On NE it requires about 34.2 s on average, whereas Dijkstra and BMSSP both stay around 0.23-0.24 s. This matches the expected growth-rate gap between $O(VE)$ and more efficient non-negative SSSP methods; on NE, Bellman-Ford is roughly 147x slower than Dijkstra and 140x slower than BMSSP. The gap is amplified by the engineering cost of scanning the full edge set on every pass, which becomes especially expensive on large road networks.

Dijkstra is the most balanced solution on these sparse road networks. It combines low runtime, low working memory, and stable source behavior; in practice, the priority queue overhead remains small relative to graph storage.

BMSSP/DMMSY remains practically competitive with Dijkstra, but it uses substantially more working memory because of its CSR layout and internal workspace buffers. This means that its speed comes with a clear **time-memory trade-off**: BMSSP stays close to Dijkstra in runtime, but uses about 1.85x the total approximate memory of Dijkstra across all three datasets.

Important caveat. BMSSP's theoretical advantage is not strongly visible here because road networks are sparse and already favorable to Dijkstra, and because the evaluated repository is a practical variant rather than a literal implementation of the paper's full theoretical machinery.

Bellman-Ford also shows clear source sensitivity: on NE the average runtime ranges from **27.8 s to 46.8 s** depending on source choice, reflecting how many relaxation passes are needed before convergence.

6. Conclusion

For large directed road-network graphs with non-negative weights, Dijkstra is the most practical overall choice: it combines low runtime, low memory cost, and stable behavior. Bellman-Ford remains the most general algorithm because it supports negative edges and cycle detection, but it is not practical at this scale. BMSSP/DMMSY achieves runtime very close to Dijkstra, yet its memory overhead is substantially higher and the tested implementation is a practical variant rather than a verbatim theoretical realization.

Table 2 summarizes the best use case, main advantages, and main disadvantages of each algorithm, providing a compact decision guide for algorithm selection in practice.

Algorithm Preference Summary

Algorithm	Best Use Case	Advantages	Disadvantages
Dijkstra	Large, sparse directed graph; all edge weights are non-negative.	Fastest overall practical baseline here; low working memory; stable across sources.	Cannot handle negative edges or negative cycles.
Bellman-Ford	Negative edges may exist, or cycle detection is required.	Most general of the three; simple relaxation idea; detects negative cycles.	By far the slowest on large graphs; runtime varies strongly with source.
BMSSP/DMMSY	High-performance directed SSSP is needed and extra memory is acceptable.	Runtime stays close to Dijkstra; scales well; source-stable in practice.	Higher working memory; practical variant of the paper; non-negative weights only.

Table 2. When each algorithm should be preferred, with key strengths and limitations.

Limitations

Memory values are analytical approximations based on data-structure sizes; allocator overhead is not captured exactly. OS `ru_maxrss` was used only as a secondary reference, not as the exact incremental memory cost of each algorithm. **The BMSSP repository used in this study is a practical variant**, not a literal implementation of all theoretical mechanisms in Duan et al. (2025); empirical results should therefore be interpreted as practical performance evidence rather than as direct confirmation of the paper's exact asymptotic bound.

References

[1] 9th DIMACS Implementation Challenge - Shortest Paths. <http://users.diag.uniroma1.it/challenge9/download.shtml>

[2] Duan, R., Mao, J., Mao, X., Shu, X., Yin, L. Breaking the Sorting Barrier for Directed Single-Source Shortest Paths. STOC 2025.

[3] DMMSY-SSSP reference implementation. <https://github.com/danalec/DMMSY-SSSP>

[4] Dijkstra, E. W. A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

[5] Bellman, R. On a Routing Problem. *Quarterly of Applied Mathematics*, 16(1):87–90, 1958.